

Answer any three of the following questions

1. Draw the block diagram of 8237 DMA controller along with associated logic and explain its role as a slave and master. 5
2. Describe briefly synchronous and asynchronous communication.
3. Illustrate how a 64 key keyboard and an 8 digit seven segment display can be connected to an 8279.
4. Define 2- key lockout and N-key rollover. Draw the general structure of the Intel 8279 Keyboard/display controller. 5

Tutorial - 3

Illustrate how a 64 key keyboard and an 8-digit seven segment display can be connected to the intel 8279 programmable keyboard/display controller.

✓ Answer:

◆ Overview of Intel 8279:

The **Intel 8279** is a programmable **keyboard and display interface** used to connect:

- A **keyboard** (like a matrix keyboard up to 64 keys),
- And a **seven-segment display** (up to 16 digits in multiplexed format).

◆ Connecting a 64-Key Keyboard:

- A 64-key matrix is typically arranged as **8 rows × 8 columns**.
- The **8279** provides:
- **SL0 to SL3** (Scan Lines) → used to scan keyboard rows.
- **RL0 to RL7** (Return Lines) → connected to columns.

✎ So, to connect 64 keys:

- Connect **SL0–SL3** to 8 **rows** through a **3-to-8 decoder** (like 74LS138).
- Connect **RL0–RL7** directly to the 8 **columns**.

 When a key is pressed, the 8279 detects it by scanning SL lines and checking which RL line responds.

◆ **Connecting an 8-Digit Seven-Segment Display:**

- Each digit has 8 segments (7 segments + dot/DP).
- **8279** provides:
- **SL0–SL3** for digit scanning.
- **OUT A0–A3** and **OUT B0–B3** for segment data.

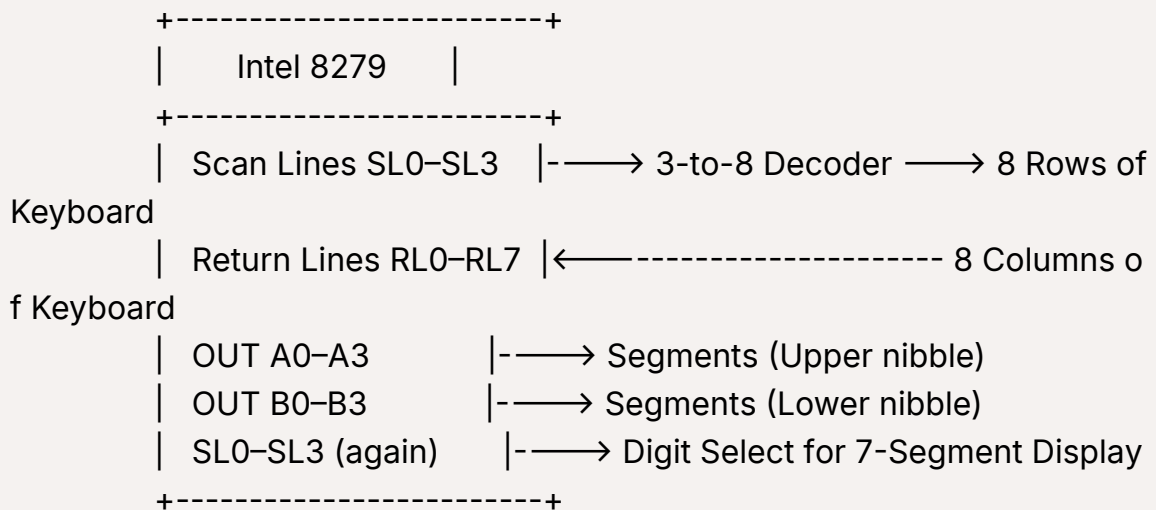
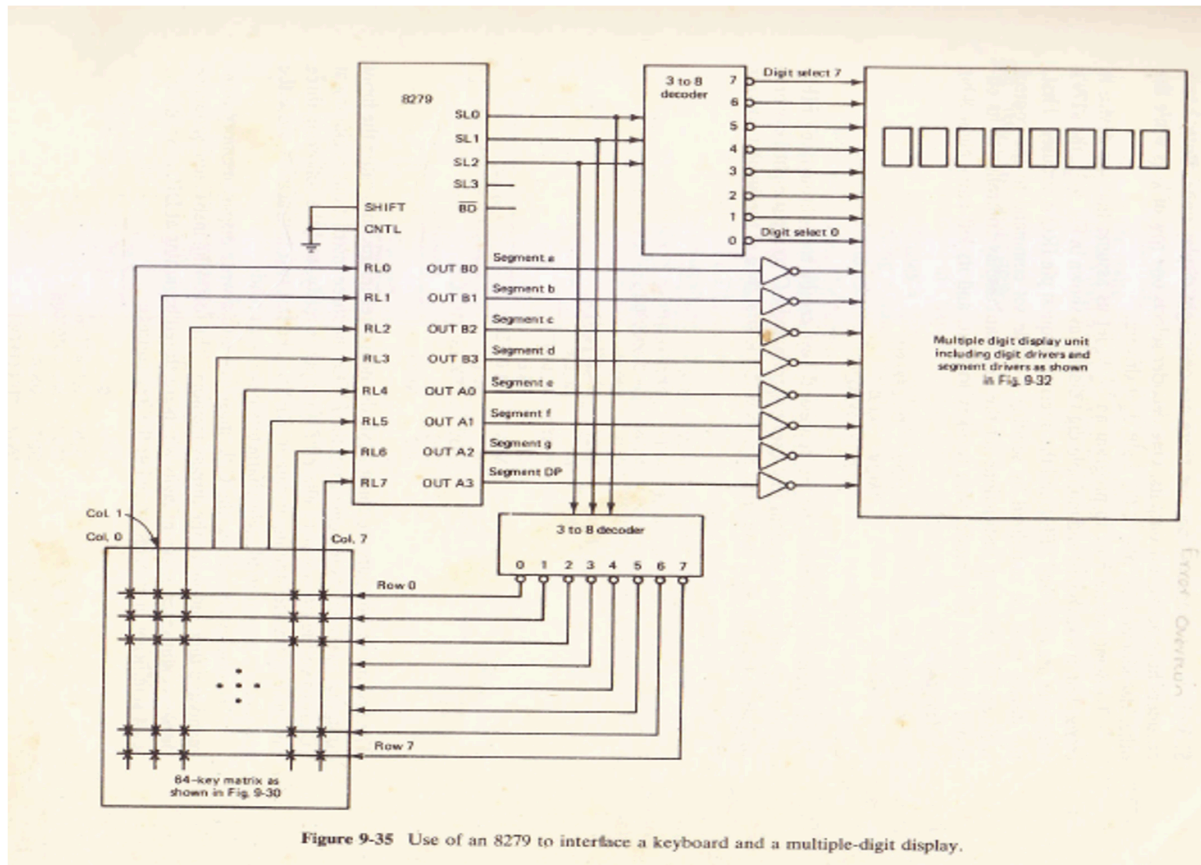
 To display 8 digits:

- Use **SL0–SL3 + internal scan counter** to multiplex and activate each digit one by one.
- Connect **OUT A0–A3** and **OUT B0–B3** to segment lines.
- OUT A → controls upper nibble of segment.
- OUT B → controls lower nibble.

 The display refreshes rapidly using time-multiplexing, so all digits appear to light up continuously.



Block Diagram to Draw in Exam:



✓ Conclusion:

By using the **scan and return lines**, the 8279 can interface with a **64-key matrix keyboard**, and by using its **output ports and scan lines**, it can multiplex and control an **8-digit seven-segment display** efficiently.

Define 2-key lockout and N-key rollover . Draw the general structure of the intel 8279 programmable keyboard/display controller.

Definitions:

1. 2-Key Lockout:

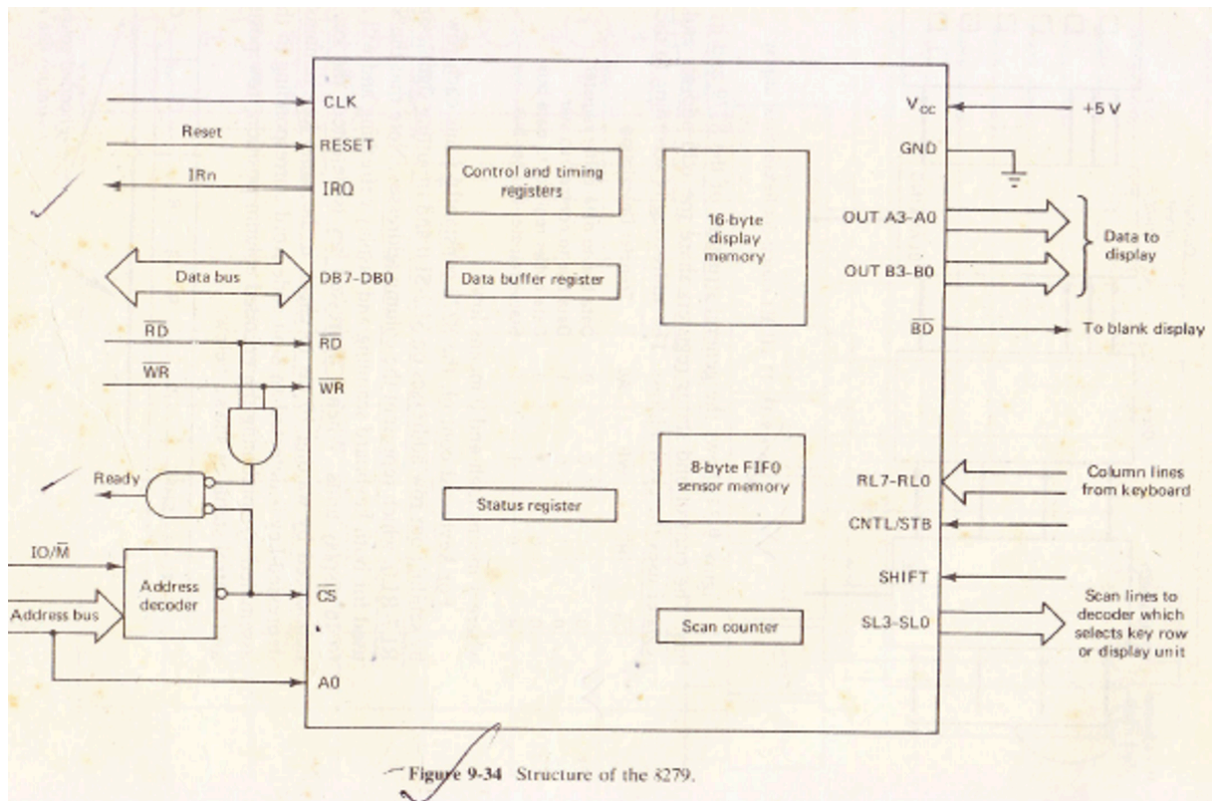
This technique allows **only one key** to be recognized at a time.

- If a second key is pressed **before** the first key is released, it is **ignored**.
- Common in **simple systems** to prevent ambiguity.

2. N-Key Rollover:

This technique allows the system to **recognize multiple simultaneous key presses**.

- All keys are **registered accurately**, even if multiple keys are pressed at once.
 - Used in **advanced keyboards** (e.g., gaming, industrial applications)
-



Functional Block Overview (Right to Left):

Display Interface:

- **OUT A3-A0 / OUT B3-B0:** Output lines that send data to the **display units** (typically 7-segment displays or LEDs).
- **BD (Blank Display):** Blanks (turns off) the display when active.
- **16-byte Display Memory:** Holds display characters. The CPU writes here; the display hardware reads from it.

Keyboard Interface:

- **RL7-RL0: Return lines** (inputs) from the keyboard matrix columns.
- **CNTL/STB:** Used to detect **key depression or external control signals**.
- **SHIFT:** Used to read shift key status (for uppercase/lowercase, etc.).
- **SL3-SL0: Scan lines** that are outputs to scan the keyboard matrix rows.

Internal Logic:

- **Scan Counter:** Cycles through scan lines for keyboard matrix and display multiplexing.
- **8-byte FIFO Sensor Memory:** Holds keystrokes in a first-in-first-out manner.
- **Status Register:** Stores the current status (e.g., key available, FIFO full, etc.).
- **Control and Timing Registers:** Used for setting up operating modes (e.g., scan rate, debounce time).

◆ **Data Communication:**

- **Data Buffer Register (DB7-DB0):** 8-bit bi-directional data bus connected to the CPU for data transfer.
- **RD, WR:** Read/Write signals from CPU to read status/data or write commands/data.
- **CS (Chip Select):** Enables communication with the chip.
- **A0:** Address line used to select **data/status** or **control**.

◆ **Other Inputs:**

- **CLK:** Clock input.
- **RESET:** Resets the internal logic.
- **IRQ:** Interrupt Request line to signal CPU when a key is pressed.
- **IRn:** Interrupt request line (typically used for keyboard input).
- **READY:** Synchronizes with CPU wait cycles.
- **I/O-M, Address Bus, Address Decoder:** Used to interface with system bus and select this chip based on address.

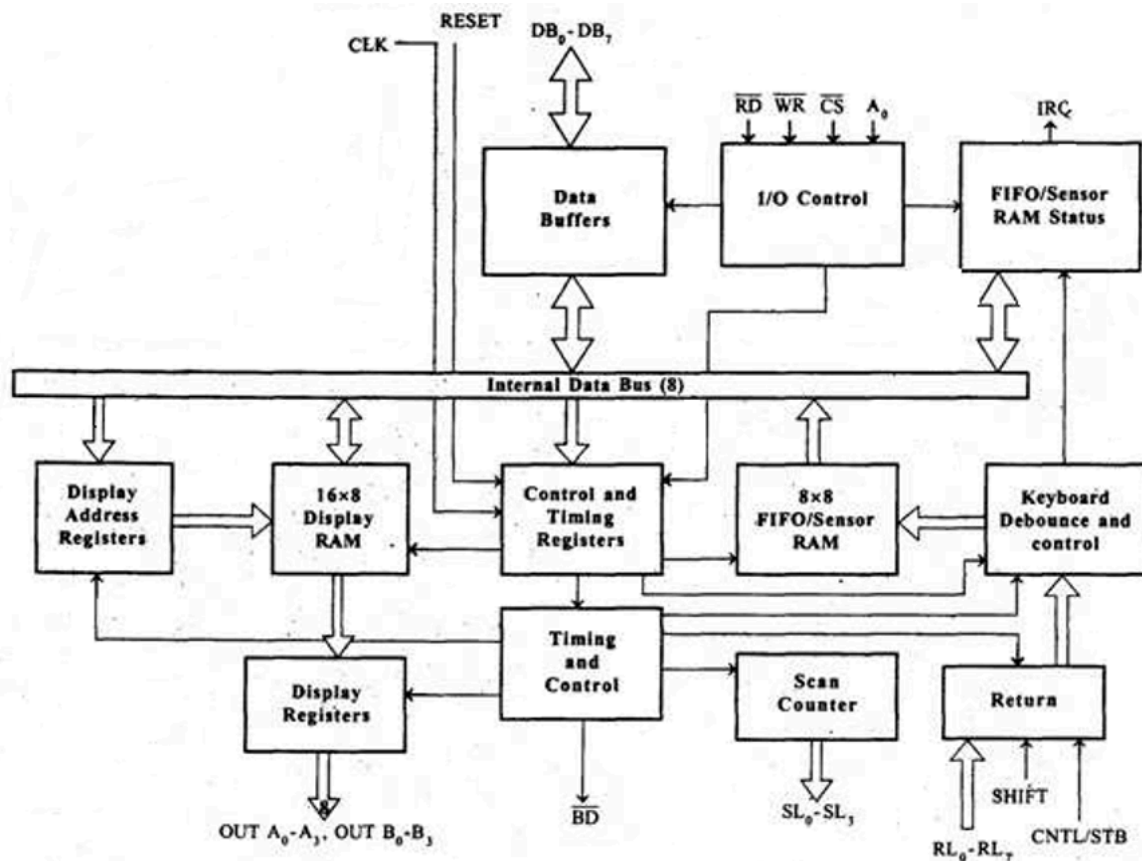
Key Functionalities:

- Scans keyboard using SL and RL lines.
- Stores key codes in FIFO buffer.
- Drives displays using multiplexing.
- Can generate interrupts to notify CPU of key events.

- Reduces CPU overhead for routine keyboard/display tasks.

Would you like a real-world example of how to program the 8279 with an 8085/8086 microprocessor?

■ Block Diagram of Intel 8279 Programmable Keyboard/Display Controller:



The diagram you shared is the **block diagram of the Intel 8279 Programmable Keyboard/Display Interface**. It is used to interface a **keyboard and display** with a **microprocessor** system efficiently.

Here's a **brief explanation of each block** in the diagram:

◆ 1. Data Buffers

- Interface the microprocessor data bus (**DB₀-DB₇**) with the internal data bus.

- Temporarily stores data being transferred between the CPU and internal registers.
-

◆ 2. I/O Control

- Controls **read (RD)**, **write (WR)**, **chip select (CS)**, and **address line (A₀)** operations.
 - Manages communication between the CPU and the 8279.
 - Selects whether the CPU is addressing data or command/status.
-

◆ 3. FIFO/Sensor RAM Status & 8×8 FIFO/Sensor RAM

- **8×8 FIFO (First-In-First-Out)** stores keystrokes temporarily.
 - Used for keyboard and sensor input buffering.
 - **Status register** gives information about the FIFO like overflow, underflow, etc.
-

◆ 4. Keyboard Debounce and Control

- Handles **debouncing** (removing unwanted repeated signals from mechanical key presses).
 - Ensures clean key press data before storing into the FIFO RAM.
-

◆ 5. Return Lines (RL₀–RL₇)

- Connected to keyboard matrix rows.
 - Used to detect keypresses.
-

◆ Scan Counter

- Generates scanning signals (**SL₀–SL₃**) to scan the keyboard and multiplexed displays.
-

◆ Timing and Control

- Generates necessary internal timing signals.
 - Synchronizes the scanning and data transfer operations.
-

◆ Control and Timing Registers

- Stores control words that define how the 8279 operates (keyboard mode, display mode, etc.).
-

◆ Display Section:

- **Display Address Registers:** Select display locations.
 - **16×8 Display RAM:** Stores data for the 7-segment displays (16 characters max, each 8-bit wide).
 - **Display Registers:** Sends display data to output lines (**OUT A₀–A₃**, **OUT B₀–B₃**).
-

◆ CNTL/STB and SHIFT Lines

- Used for **special function key** recognition.
 - CNTL/STB can act as a control/strobe signal for programmable functions.
-

◆ IRC (Interrupt Request)

- Sends an **interrupt signal** to the microprocessor when a key is pressed or FIFO is ready.
-

✓ Summary:

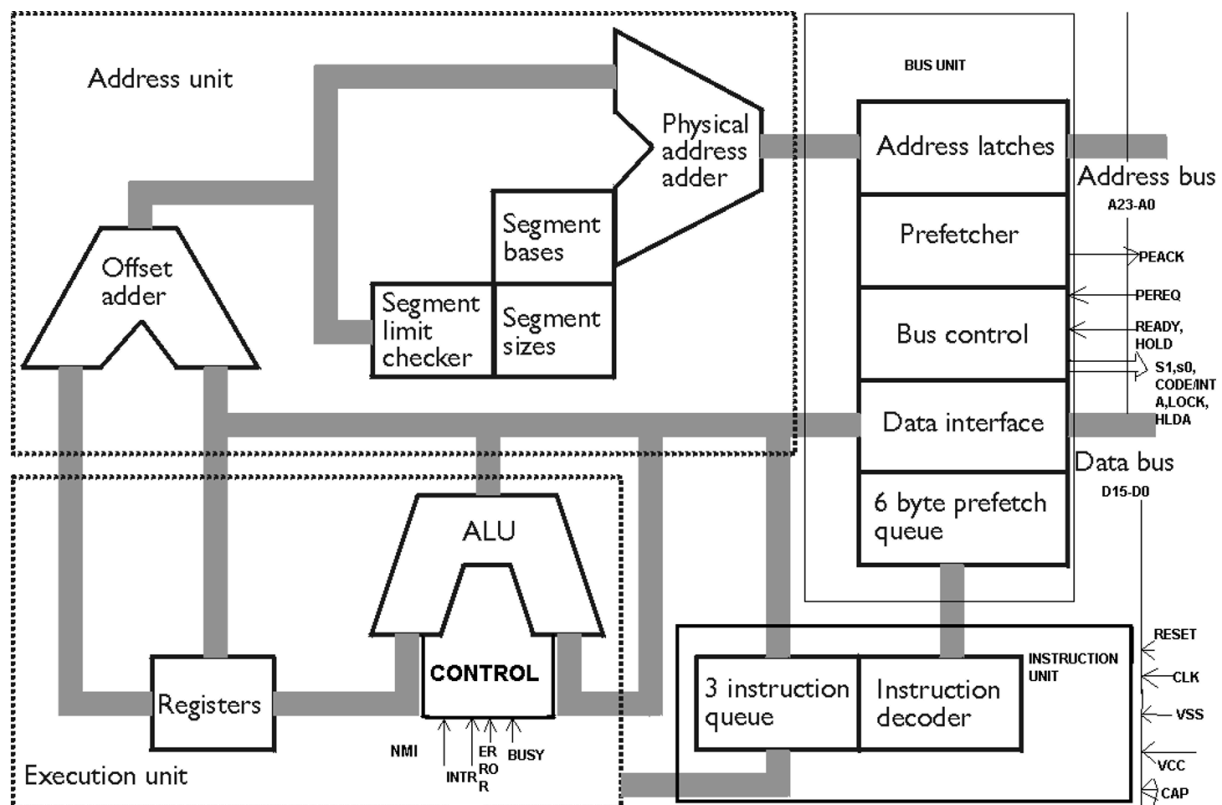
The **Intel 8279**:

- Interfaces a **64-key keyboard** and **16-character display**.
- Reduces CPU load by handling keyboard scanning, debouncing, display refreshing, and buffering independently.
- Communicates with the CPU through simple commands and data transfer operations.

Let me know if you want a **simplified hand-drawn version** for your exam prep!

Let me know if you'd like a simpler version to sketch quickly in the exam or need help understanding any block!

Draw and describe the internal architecture of 80286 microprocessor.
Write what are programme invisible registers



The image above illustrates the **internal architecture of the Intel 80286 microprocessor**. It divides the processor into key functional units that work together to execute instructions efficiently.

Key Functional Blocks:

1. Execution Unit (EU):

- **Registers:** Temporary storage for operands and results (e.g., AX, BX, etc.).
- **ALU (Arithmetic Logic Unit):** Performs arithmetic and logical operations.
- **Control Unit:** Directs data flow and coordinates all operations.
- **Instruction Queue (3 bytes):** Buffers instructions to be executed.
- Connected to the **instruction decoder** to interpret opcodes.

2. Instruction Unit (IU):

- **Instruction Decoder:** Decodes fetched instructions into micro-operations.
- Works with the **3-byte instruction queue** for pipelining and speed.

3. Address Unit (AU):

- **Offset Adder:** Combines offsets and displacement to form logical addresses.
 - **Segment Limit Checker:** Ensures accesses are within segment boundaries.
 - **Segment Bases and Sizes:** Part of the memory segmentation system.
 - **Physical Address Adder:** Combines segment base and offset to form a physical address.
-

4. Bus Unit:

- Interfaces with external components via the **address and data buses**.
 - **Address Latches:** Hold the output address lines A23–A0.
 - **Prefetcher:** Fetches instructions from memory in advance to improve performance.
 - **6-byte Prefetch Queue:** Buffers instructions before they are decoded.
 - **Bus Control & Data Interface:** Handles bus arbitration and data transfers.
-

External Signals and Buses:

- **Address Bus (A23–A0):** Carries memory and I/O addresses.
 - **Data Bus (D15–D0):** Carries data between processor and memory or I/O.
 - **Control Signals:** Includes **RD**, **WR**, **READY**, **RESET**, **CLK**, etc.
 - **Interrupts:** **INTR**, **NMI**, and **ERROR** are shown for interrupt handling.
-

Summary:

This architecture allows the 80286 to:

- Support **segmented memory** with **protected mode**,
- Achieve **instruction pipelining** using prefetch queues,
- Perform **address translation and checking** for memory protection,
- Efficiently execute instructions using the **ALU and registers**.

Would you like a simplified version of this diagram or a comparison with another processor like 8086?

Program-Invisible Registers

Program-invisible registers are internal registers within the processor that are **not directly accessible or modifiable by user-level programs or instructions**. These registers play a vital role in supporting **memory management** and **segmentation**, especially in advanced microprocessors like the **80286**, **80386**, and later x86 CPUs.

Purpose:

Program-invisible registers are used by the CPU to:

- Manage **segmentation** in protected mode.
- Store **descriptor table entries** for each segment (base, limit, access rights).
- Speed up memory access by **caching segment descriptors**.

Examples of Program-Invisible Registers (in 80286/80386):

For each **segment register** (e.g., **CS**, **DS**, **SS**, **ES**, **FS**, **GS**), there are associated program-invisible registers:

1. **Segment Base Address** – The starting physical address of the segment.
2. **Segment Limit** – Defines the size of the segment.
3. **Access Rights** – Defines permissions (read/write, privilege level, etc.).

How They Work:

When a segment register is loaded (e.g., with a **MOV** instruction), the CPU:

1. Uses the segment selector to locate the **descriptor** in the **GDT** or **LDT**.
2. Loads the **base**, **limit**, and **access rights** into program-invisible registers.
3. These registers are then used for **address translation** and **protection checking** without additional memory lookups.

Why "Invisible"?

Because:

- They **cannot be accessed or changed directly** by software.
 - Only the **CPU's internal mechanisms** interact with them.
-

✅ Summary:

Program-invisible registers are essential for efficient **segmentation and protection** in advanced x86 processors, supporting the CPU's ability to translate and validate memory accesses **transparently to the programmer**.

Would you like a diagram showing how these registers interact with the GDT/LDT and segment registers?

Draw the block diagram of 8237 DMA controller along with associated logic and explain it's role as a slave and master

Below is the block diagram of the Intel 8237 DMA (Direct Memory Access) Controller:

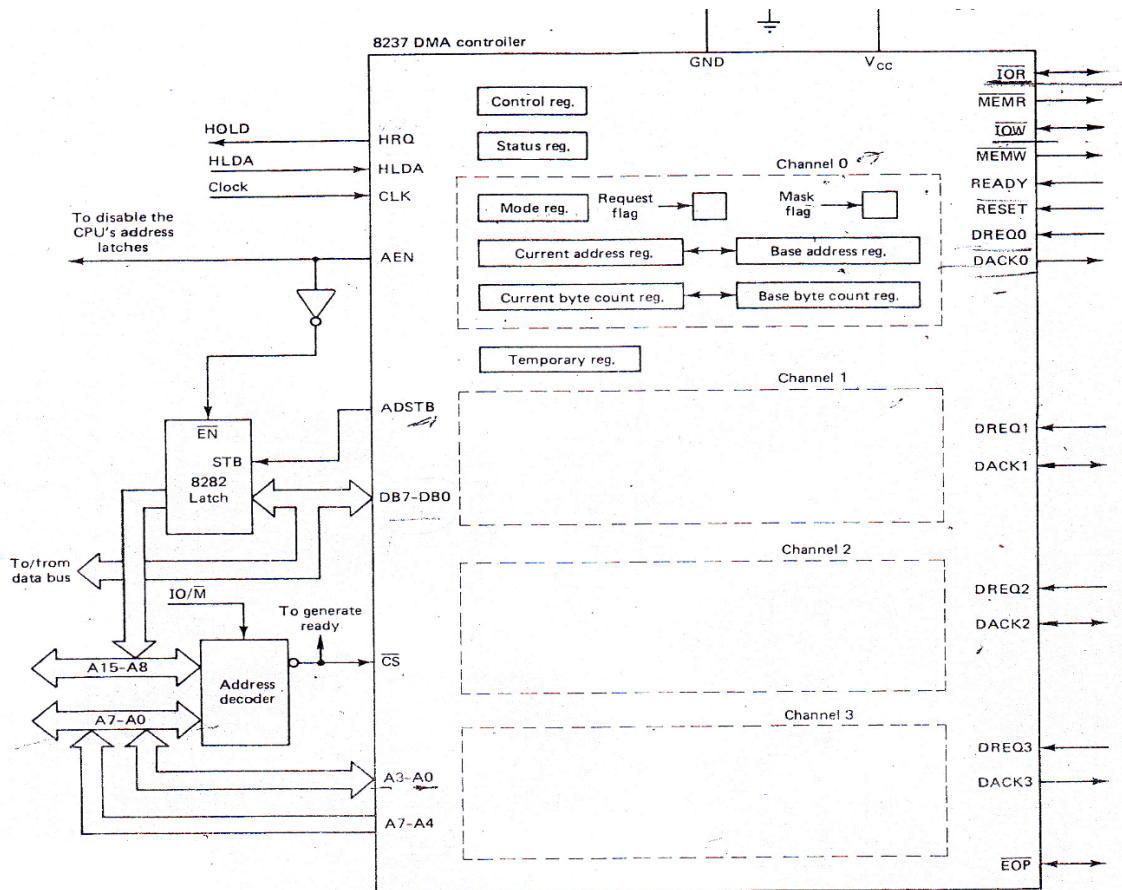


Figure 9-37 Organization of an 8237 and its associated logic.

The provided diagram shows the **8237 DMA (Direct Memory Access) Controller** and its **associated logic**. The 8237 allows peripherals to directly read from or write to memory without continuous CPU intervention, significantly increasing data transfer efficiency.

Block Diagram of 8237 DMA Controller (with Associated Logic)

The diagram consists of:

- The **8237 DMA controller chip**
- External components like **latches, address decoder, and control lines**
- **4 DMA channels** (Channel 0 to Channel 3)
- Control/status/data registers
- Interfacing signals with CPU and memory

Explanation of Each Section

◆ 1. Control and Status Logic

- **Control Register:** Used to set up operations (e.g., read/write/mode selection).
 - **Status Register:** Reports current status (e.g., which channel is active).
 - **Mode Register:** Specifies the transfer type (read/write, block, demand, etc.).
-

◆ 2. Channel Blocks (0–3)

Each channel has:

- **Current Address Register:** Keeps the memory address to/from which data is transferred.
 - **Current Byte Count Register:** Stores how many bytes to transfer.
 - **Base Address/Byte Count Registers:** Backup of initial values.
 - **Request and Mask Flags:** Control whether the channel is enabled and when it can request DMA.
-

◆ 3. Associated Logic

- **Address Decoder:** Decodes address lines A0–A15 to select internal DMA registers.
 - **8282 Latch:** Latches the upper byte of the address (A15–A8) during DMA operations.
 - **Clock, RESET, READY:** Timing and synchronization signals with CPU.
-

◆ 4. Data and Control Lines

- **DB7–DB0:** Bi-directional data bus for communication with CPU.
- **HRQ (Hold Request):** Sent to CPU requesting control of system buses.
- **HLDA (Hold Acknowledge):** Sent by CPU when it gives up control.
- **IOR/MEMR/IOW/MEMW:** Controls for reading/writing from/to I/O or memory.

- **DREQx/DACKx**: Peripheral request and DMA acknowledgment lines.
 - **EOP**: End of process signal.
-



Role of 8237 as Master and Slave

As a Slave:

- During **initialization**, the CPU programs the DMA controller.
- 8237 listens to CPU commands through address and data lines.
- It receives mode, address, and byte count configurations.
- **Control signals like IOR, IOW, A0–A3** are used.

 *In this mode, the 8237 acts like a peripheral device responding to the CPU.*

As a Master:

- After receiving a **DMA request (DREQx)**, the 8237 sends **HRQ** to the CPU.
- When CPU responds with **HLDA**, 8237 takes control of the **system buses**.
- Performs memory read/write cycles independently using **MEMR, MEMW, IOR, IOW**.
- Returns control to CPU after finishing the data transfer.

 *In this mode, 8237 behaves as a bus master, performing direct memory operations.*

Summary

- The **8237 DMA controller** enables fast and efficient data transfers.
- As a **slave**, it's configured by the CPU.
- As a **master**, it temporarily takes over the buses to directly handle data between peripherals and memory.

Would you like a simplified diagram version for easier memorization or exam prep?